

Pontificia Universidad Católica de Chile

EPG4001 Aprendizaje Supervisado

Material Complementario

Clase4MIAApSup.pdf

Jorge Luis Bazán
jlbazan@uc.cl

Librerías

Las librerías a utilizar para los procedimientos son las siguientes

```
library(ISLR)      # Bases de datos del libro ISL
library(tidyverse) # Manipulación y visualización
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr    1.6.0
## v lubridate  1.9.5      v tibble     3.3.1
## v purrr      1.2.2      v tidyr      1.3.2
## -- Conflicts ----- tidyverse_conflicts() --
## x ggplot2::element() masks e1071::element()
## x dplyr::filter()     masks stats::filter()
## x dplyr::lag()        masks stats::lag()
## x purrr::lift()       masks caret::lift()
## x dplyr::select()     masks MASS::select()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)      # K-Fold y métricas generales
library(pROC)        # Curva ROC
library(boot)        # Bootstrap y cv.glm
```

```
##
## Attaching package: 'boot'
##
## The following objects are masked from 'package:faraway':
##
##   logit, melanoma
##
## The following object is masked from 'package:lattice':
##
##   melanoma
```

```
library(Metrics)    # Métricas: F1, precision, recall, RMSE, etc.
```

```
##
## Attaching package: 'Metrics'
##
## The following objects are masked from 'package:caret':
##
##   precision, recall
##
## The following object is masked from 'package:pROC':
##
##   auc
```

```
library(knitr)       # Tablas
```

1. Regresión Logística: Default

1.1. Base de datos

La base `Default` contiene información de **10,000 clientes** de tarjetas de crédito. El objetivo es predecir si un cliente incurrirá en **default** (impago): $y_i \in \{0, 1\}$.

Cargando y explorando la data:

```
data(Default)
glimpse(Default)

## Rows: 10,000
## Columns: 4
## $ default <fct> No, No, No, No, No, No, No, No, No, No, No, No, No, No, No, No, No, No, No, No~
## $ student <fct> No, Yes, No, No, No, Yes, No, Yes, No, No, Yes, Yes, No, No, No, No, No, No, No, No~
## $ balance <dbl> 729.5265, 817.1804, 1073.5492, 529.2506, 785.6559, 919.5885, 8~
## $ income <dbl> 44361.625, 12106.135, 31767.139, 35704.494, 38463.496, 7491.55~

Default %>%
  count(default) %>%
  mutate(proporcion = round(n / sum(n), 3)) %>%
  kable(caption = "Distribución de la variable Default",
        booktabs = TRUE)
```

Cuadro 1: Distribución de la variable Default

default	n	proporcion
No	9667	0.967
Yes	333	0.033

Vemos que la data está desbalanceada, siendo solo el 3.3 % casos de impago.

```
ggplot(Default, aes(x = balance, y = income, color = default)) +
  geom_point(alpha = 0.3, size = 0.7) +
  scale_color_manual(values = c("steelblue", "tomato")) +
  labs(x = "Balance", y = "Income", color = "Default") +
  theme_minimal(base_size = 11)
```

A partir del scatterplot entre Balance e Income no se ven patrones fuertes de relación entre estas variables. Sí observamos que para quienes incurrn en default se agrupan en valores mayores de Balance.

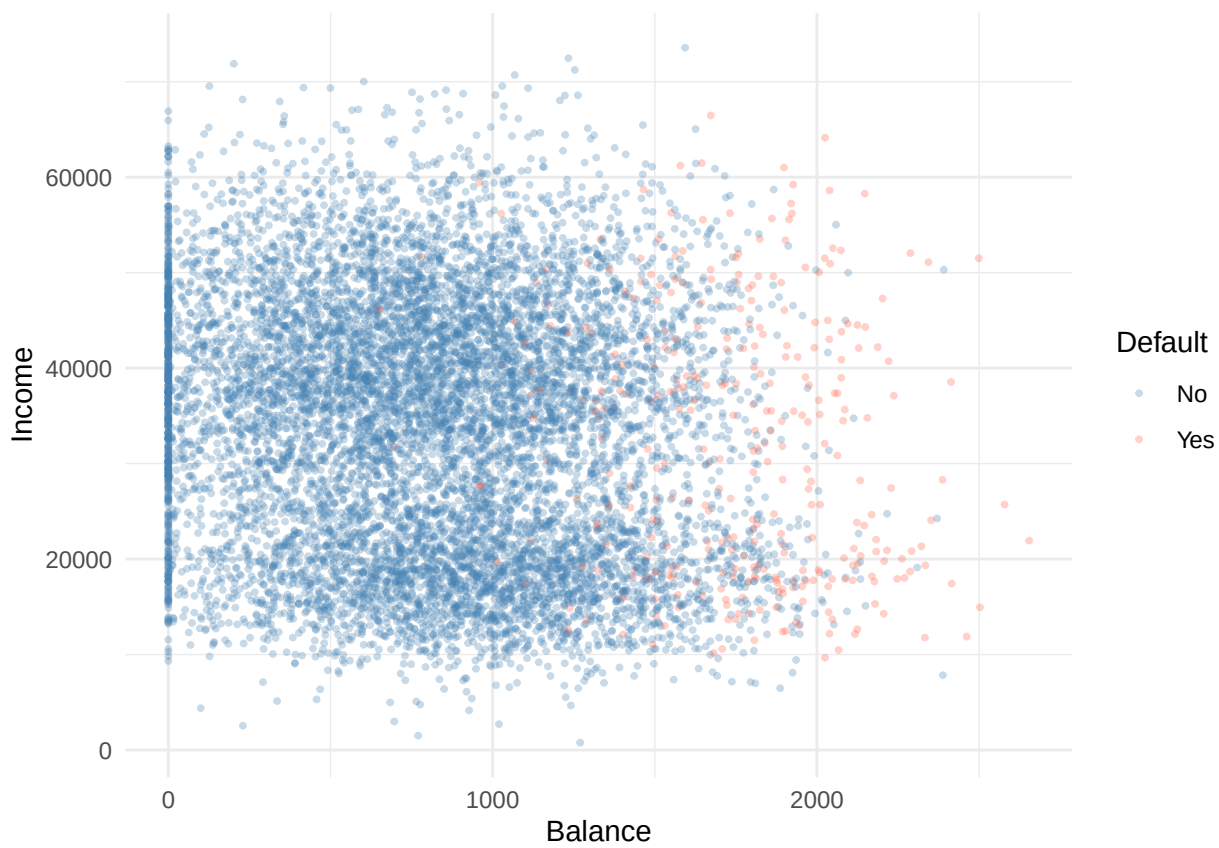


Figura 1: Balance vs Income según Default

1.2. Ajuste del modelo de Regresión Logística

El modelo logístico es:

$$y_i \sim \text{Bernoulli}(\pi_i)$$
$$\text{logit}(\pi_i) = \beta_0 + \beta_1 \text{balance}_i + \beta_2 \text{income}_i + \beta_3 \text{student}_i$$

Traspassando los valores de la variable a 0 y 1:

```
Default <- Default %>%  
  mutate(default_bin = ifelse(default == "Yes", 1, 0))
```

Ajustamos el modelo con el comando `glm` y obtenemos un resumen de sus resultados:

```
modelo_logit <- glm(  
  default ~ balance + income + student,  
  data = Default,  
  family = binomial(link = "logit")  
)  
  
summary(modelo_logit)  
  
##  
## Call:  
## glm(formula = default ~ balance + income + student, family = binomial(link = "logit"),  
##      data = Default)  
##  
## Coefficients:  
##              Estimate Std. Error z value Pr(>|z|)  
## (Intercept) -1.087e+01  4.923e-01 -22.080  < 2e-16 ***  
## balance      5.737e-03  2.319e-04  24.738  < 2e-16 ***  
## income       3.033e-06  8.203e-06   0.370  0.71152  
## studentYes  -6.468e-01  2.363e-01  -2.738  0.00619 **  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for binomial family taken to be 1)  
##  
##      Null deviance: 2920.6  on 9999  degrees of freedom  
## Residual deviance: 1571.5  on 9996  degrees of freedom  
## AIC: 1579.5  
##  
## Number of Fisher Scoring iterations: 8
```

Vemos que en términos de significancia las variables `balance` y si el sujeto es o no estudiante resultan significativas con un 95 % de confianza para realizar las predicciones.

1.2.1 Clasificación y Matriz de Confusión

Obtenemos primero las probabilidades predichas por el modelo y a partir de ellas catalogamos a cada sujeto considerando primeramente un threshold de 0,5.

```
y<- Default$default_bin
pi_hat <- fitted(modelo_logit)
y_pred <- factor(ifelse(pi_hat >= 0.5, "Yes", "No"), levels = c("No", "Yes"))
```

A partir de estas clasificaciones obtenemos ya la matriz de confusión

```
conf_mat <- confusionMatrix(y_pred, Default$default, positive = "Yes")
print(conf_mat)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction   No  Yes
##      No  9627  228
##      Yes   40  105
##
##              Accuracy : 0.9732
##              95% CI : (0.9698, 0.9763)
##      No Information Rate : 0.9667
##      P-Value [Acc > NIR] : 0.0001044
##
##              Kappa : 0.4278
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##              Sensitivity : 0.3153
##              Specificity : 0.9959
##              Pos Pred Value : 0.7241
##              Neg Pred Value : 0.9769
##              Prevalence : 0.0333
##              Detection Rate : 0.0105
##      Detection Prevalence : 0.0145
##              Balanced Accuracy : 0.6556
##
##      'Positive' Class : Yes
##
```

1.2.2. F1 Score

Podemos calcular el F1 Score para el modelo ajustado

$$F_1 = \frac{2VP}{2VP + FP + FN}$$

```
VP <- conf_mat$table[2, 2]
FP <- conf_mat$table[2, 1]
FN <- conf_mat$table[1, 2]

cat("F1 Score:", round(2*VP/(2*VP+FP+FN), 4), "\n")
```

```
## F1 Score: 0.4393
```

Usando la librería *Metrics*:

```
precision_val <- Metrics::precision(y, as.numeric(y_pred == "Yes"))
recall_val    <- Metrics::recall(y,   as.numeric(y_pred == "Yes"))
cat("Precision:", round(precision_val, 4), "\n")
```

```
## Precision: 0.7241
```

```
cat("Especificidad:", round(recall_val, 4), "\n")
```

```
## Especificidad: 0.3153
```

Vemos que, empleando un threshold de 0.5 la precisión resultó ser de 0.72, mucho mejor que la especificidad de 0.32, esto sería indicio que el modelo tiene mejor desempeño identificando verdaderos sujetos que caigan en default que dejando fuera a quienes no presentan default realmente. Podemos obtener la curva ROC y el threshold óptimo:

```
roc_obj <- roc(Default$default_bin, pi_hat, quiet = TRUE)

# Find optimal threshold (maximizes Sensitivity + Specificity)
coords_opt <- coords(roc_obj, x = "best", best.method = "youden",
                     ret = c("threshold", "sensitivity", "specificity"))

cat("Umbral óptimo: ", round(coords_opt$threshold, 4), "\n")
```

```
## Umbral óptimo: 0.0312
```

```
cat("Sensibilidad: ", round(coords_opt$sensitivity, 4), "\n")
```

```
## Sensibilidad: 0.9039
```

```
cat("Especificidad: ", round(coords_opt$specificity, 4), "\n")
```

```
## Especificidad: 0.8612
```

```
# Plot
plot(roc_obj, col = "tomato", lwd = 2,
     main = paste("Curva ROC | AUC =", round(pROC::auc(roc_obj), 3)))
abline(a = 0, b = 1, lty = 2, col = "gray60")

# Add optimal point
points(
  x = 1-coords_opt$specificity, # FPR = 1 - Specificity
  y = coords_opt$sensitivity,
  pch = 19, col = "steelblue", cex = 2
)
text(
  x = 1-coords_opt$specificity,
  y = coords_opt$sensitivity,
  labels = paste0("Umbral = ", round(coords_opt$threshold, 3)),
  pos = 4, cex = 0.85, col = "steelblue"
)
```

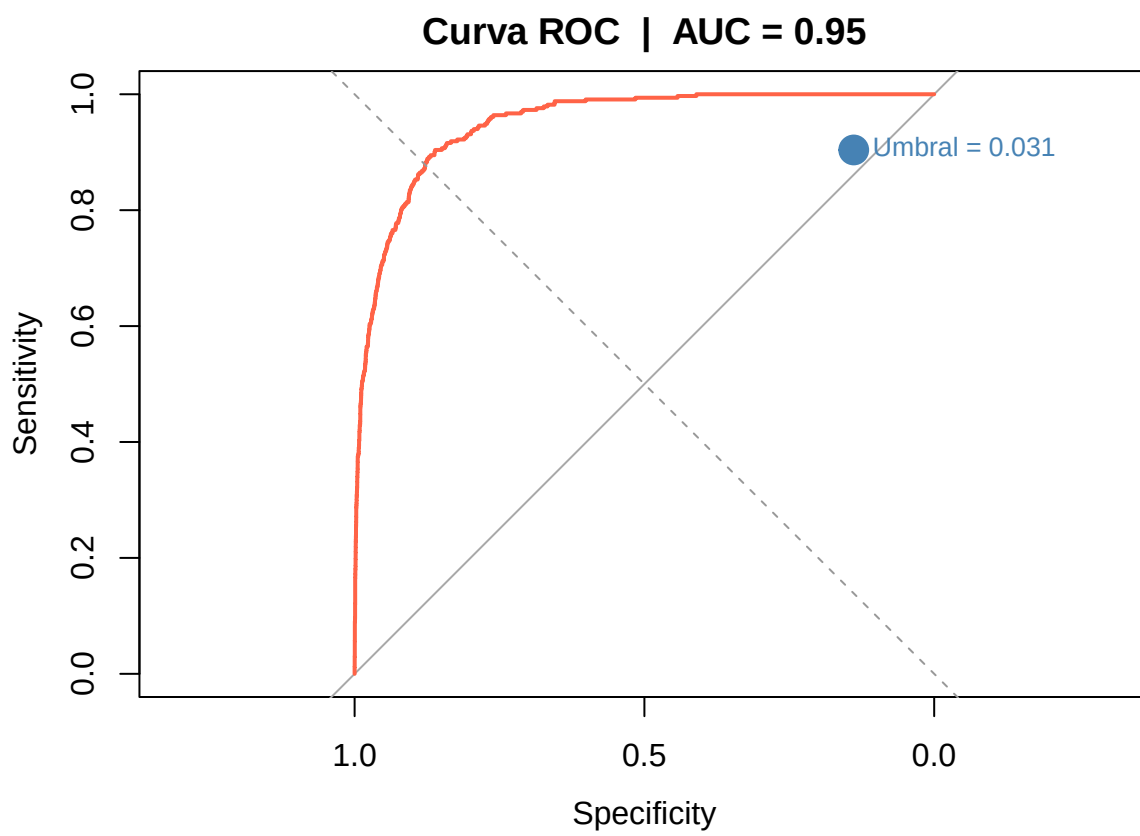


Figura 2: Curva ROC — Regresión Logística (Default)

El AUC resultante es de 0.95 para el punto óptimo umbral de $1 - 0,031 = 0,969$, esto es el modelo clasificará a los sujetos como “default” para valores de probabilidad sobre 0.031 y con este punto de corte se obtiene una sensibilidad de 0.9 y especificidad de 0.86.

A continuación, considerando como función de pérdida la Devianza media ($L(Y, f(\cdot)) = D/n = -2 \log(\hat{p}_Y)/n = -2\ell/n$), de esta manera los resultados obtenidos por los distintos métodos serán comparables (LOO y Bootstrap632 obtienen valor de error estimado por observación). Realizando las estimaciones:

1.3.1. Validación Cruzada Simple

Dividimos el conjunto de datos en **70 % entrenamiento** y **30 % prueba**, para esto seleccionamos aleatoriamente los casos con `sample()` y separamos la base en `train_d` (data de entrenamiento) y `test_d` (data de test):

```
set.seed(123)
n <- nrow(Default)
idx <- sample(1:n, size = floor(0.7 * n))
train_d <- Default[idx, ]
test_d <- Default[-idx, ]
```

Ajustamos el modelo con la data de entrenamiento:

```
mod_hold <- glm(default ~ balance + income + student,
  data = train_d, family = binomial())
```

Predecimos y evaluamos con la data de test:

```
pi_hold <- predict(mod_hold, newdata = test_d, type = "response")
y_hold <- factor(ifelse(pi_hold >= 0.5, "Yes", "No"), levels = c("No", "Yes"))
cm_hold <- confusionMatrix(y_hold, test_d$default, positive = "Yes")

cat("Accuracy: ", round(cm_hold$overall["Accuracy"], 4), "\n")
```

```
## Accuracy: 0.973
```

```
cat("F1 Score: ", round(cm_hold$byClass["F1"], 4), "\n")
```

```
## F1 Score: 0.4255
```

Vemos que utilizando validación cruzada simple ahora se obtiene un puntaje F1 Score de 0.425 y accuracy (especificidad) de 0.973.

1.3.2. Validación Cruzada K-Fold

La librería `caret` permite realizar K-Fold de forma directa, dividimos el conjunto en **K = 10 pliegues**.

```
set.seed(123)
# Función summary personalizada: ROC + Mean Deviance
custom_summary <- function(data, lev = NULL, model = NULL) {
  # Métricas estándar de twoClassSummary
```

```

tcs <- twoClassSummary(data, lev, model)
# Probabilidades de la clase positiva ("Yes")
p <- data$Yes
y <- as.numeric(data$obs == "Yes")
# Mean Deviance
mdev <- -2 * mean(y * log(p + 1e-15) + (1 - y) * log(1 - p + 1e-15))
c(tcs, MeanDeviance = mdev)
}

ctrl_kfold <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = custom_summary,
  savePredictions = "final"
)

```

Ajustando en el entrenamiento empleando la métrica de curva ROC:

```

mod_kfold <- train(
  default ~ balance + income + student,
  data = Default,
  method = "glm",
  family = binomial(),
  trControl = ctrl_kfold,
  metric = "ROC"
)

```

Obteniendo los resultados:

```

# Resultados por pliegue
kfold_res <- mod_kfold$resample
kable(kfold_res, digits = 4,
      caption = "Métricas por pliegue - K-Fold (K=10, caret)",
      booktabs = TRUE)

```

Cuadro 2: Métricas por pliegue — K-Fold (K=10, caret)

ROC	Sens	Spec	MeanDeviance	Resample
0.9696	0.9959	0.4242	0.1368	Fold01
0.9393	0.9948	0.3235	0.1705	Fold02
0.9539	0.9938	0.3939	0.1552	Fold03
0.9397	0.9948	0.4242	0.1491	Fold04
0.9348	0.9969	0.2727	0.1650	Fold05
0.9491	0.9979	0.3030	0.1581	Fold06
0.9470	0.9979	0.2059	0.1798	Fold07
0.9567	0.9969	0.2727	0.1439	Fold08
0.9459	0.9917	0.2727	0.1710	Fold09
0.9575	0.9969	0.3235	0.1496	Fold10

```
cat("\nPromedios K-Fold:\n")
```

```
##
## Promedios K-Fold:
```

```
colMeans(kfold_res[, c("ROC", "Sens", "Spec", "MeanDeviance")]) %>%
  round(4) %>% print()
```

```
##          ROC          Sens          Spec MeanDeviance
##          0.9494          0.9958          0.3217          0.1579
```

La devianza media con validación cruzada simple resultó ser de 0.1579.

1.3.3. Validación Cruzada Aleatoria (Bootstrap)

Se consideran tres estimadores del error de predicción:

1.3.3.1. Estimador Bootstrap estándar

Si $\hat{f}^{*b}(x_i)$ es el valor predicho a partir del modelo ajustado al b -ésimo conjunto bootstrap, el estimador del error es:

$$\widehat{\text{Err}}_{boot} = \frac{1}{B} \frac{1}{N} \sum_{b=1}^B \sum_{i=1}^N L(y_i, \hat{f}^{*b}(x_i))$$

Seteamos semilla y definimos el número de muestras bootstrap:

```
set.seed(42)
B <- 200 # número de muestras bootstrap
```

Aplicamos el método ajustando con `glm()`:

```
err_boot_std <- replicate(B, {
  idx_b <- sample(1:n, size = n, replace = TRUE)
  train_b <- Default[idx_b, ]
  mod_b <- glm(default ~ balance + income + student,
               data = train_b, family = binomial())
  p_b <- predict(mod_b, newdata = Default, type = "response")
  # Deviance = -2 * log-likelihood
  -2 * mean(Default$default_bin * log(p_b + 1e-15) +
            (1 - Default$default_bin) * log(1 - p_b + 1e-15))
})

cat("Err_boot Deviance (estándar):", round(mean(err_boot_std), 4), "\n")
```

```
## Err_boot Deviance (estándar): 0.1576
```

```
cat("SD: ", round(sd(err_boot_std), 4), "\n")
```

```
## SD: 3e-04
```

1.3.3.2. Estimador Leave-One-Out Bootstrap $\widehat{\text{Err}}^{(1)}$

Para corregir el sesgo, se evalúa cada observación i **solo** en las muestras bootstrap que **no** la contienen:

$$\widehat{\text{Err}}^{(1)} = \frac{1}{N} \sum_{i=1}^N \frac{1}{|C^{(-i)}|} \sum_{b \in C^{(-i)}} L(y_i, \hat{f}^{*b}(x_i))$$

donde $C^{(-i)}$ es el conjunto de muestras bootstrap que no incluyen i .

```
set.seed(42)
B <- 200

preds_matrix <- matrix(NA, nrow = n, ncol = B)
in_bag <- matrix(FALSE, nrow = n, ncol = B)

for (b in seq_len(B)) {
  idx_b <- sample(1:n, size = n, replace = TRUE)
  in_bag[idx_b, b] <- TRUE
  mod_b <- glm(default ~ balance + income + student,
               data = Default[idx_b, ],
               family = binomial())
  preds_matrix[, b] <- predict(mod_b, newdata = Default, type = "response")
}

# Para cada observación, promediar solo los b donde i NO estaba
loo_errors <- sapply(1:n, function(i) {
  oob <- which(!in_bag[i, ])
  if (length(oob) == 0) return(NA)
  p_oob <- preds_matrix[i, oob]
  # Deviance individual: -2 * log-likelihood para obs i
  mean(-2 * (Default$default_bin[i] * log(p_oob + 1e-15) +
            (1 - Default$default_bin[i]) * log(1 - p_oob + 1e-15)))
})

err_loo <- mean(loo_errors, na.rm = TRUE)
cat("Err^(1) L00 Deviance:", round(err_loo, 4), "\n")
```

```
## Err^(1) L00 Deviance: 0.1583
```

1.3.3.3. Estimador 0.632 Bootstrap $\widehat{\text{Err}}^{(0,632)}$

Para corregir el sesgo de $\widehat{\text{Err}}^{(1)}$, se pondera con el error aparente $\overline{\text{err}}$ (error en entrenamiento completo):

$$\widehat{\text{Err}}^{(0,632)} = 0,368 \cdot \overline{\text{err}} + 0,632 \cdot \widehat{\text{Err}}^{(1)}$$

```
# Número de observaciones
n <- nrow(Default)

# Error aparente en escala mean deviance
err_apparent <- deviance(modelo_logit) / n

err_632 <- 0.368 * err_apparent + 0.632 * err_loo

cat("Deviance aparente (err_bar): ", round(err_apparent, 4), "\n")
```

```
## Deviance aparente (err_bar): 0.1572
```

```
cat("Err^(1) L00 Deviance: ", round(err_loo, 4), "\n")
```

```
## Err^(1) L00 Deviance: 0.1583
```

```
cat("Err^(0.632) Deviance: ", round(err_632, 4), "\n")
```

```
## Err^(0.632) Deviance: 0.1579
```

1.3.4. Función directa `boot::cv.glm` (K-Fold desde GLM)

La función `cv.glm` del paquete `boot` permite hacer **K-Fold** directamente sobre objetos `glm`, usando la devianza:

```
# Función de costo: Deviance = -2 * log-likelihood
costo_logit <- function(y, p) {
  -2 * sum(y * log(p + 1e-15) + (1 - y) * log(1 - p + 1e-15))
}

set.seed(123)
# LOOCV (K = n)
cv_loocv <- cv.glm(Default, modelo_logit, cost = costo_logit, K = n)
# 10-Fold
cv_10fold <- cv.glm(Default, modelo_logit, cost = costo_logit, K = 10)

cat("cv.glm LOOCV - delta[1] (raw): ", round(cv_loocv$delta[1], 4), "\n")
```

```
## cv.glm LOOCV - delta[1] (raw): 0.158
```

```
cat("cv.glm LOOCV - delta[2] (bias-corrected): ", round(cv_loocv$delta[2], 4), "\n")
```

```
## cv.glm LOOCV - delta[2] (bias-corrected): 0.1576
```

```
cat("cv.glm 10-Fold - delta[1] (raw): ", round(cv_10fold$delta[1], 4), "\n")
```

```
## cv.glm 10-Fold - delta[1] (raw): 157.7981
```

```
cat("cv.glm 10-Fold - delta[2] (bias-corrected): ", round(cv_10fold$delta[2], 4), "\n")
```

```
## cv.glm 10-Fold - delta[2] (bias-corrected): 157.459
```

Vemos que los distintos criterios arrojan valores de errores de predicción medios similares, cercanos a una devianza media de 0.15

2. Regresión Múltiple

2.1. Base de datos: Advertising

La base Advertising contiene datos de **200 mercados**. El objetivo es predecir **sales** (ventas) a partir del gasto en publicidad en TV, Radio y Newspaper.

```
Advertising <- read.csv(
  "https://www.statlearning.com/s/Advertising.csv",
  row.names = 1
)
glimpse(Advertising)

## Rows: 200
## Columns: 4
## $ TV      <dbl> 230.1, 44.5, 17.2, 151.5, 180.8, 8.7, 57.5, 120.2, 8.6, 199.~
## $ radio    <dbl> 37.8, 39.3, 45.9, 41.3, 10.8, 48.9, 32.8, 19.6, 2.1, 2.6, 5.~
## $ newspaper <dbl> 69.2, 45.1, 69.3, 58.5, 58.4, 75.0, 23.5, 11.6, 1.0, 21.2, 2~
## $ sales    <dbl> 22.1, 10.4, 9.3, 18.5, 12.9, 7.2, 11.8, 13.2, 4.8, 10.6, 8.6~
```

2.1. Ajuste del modelo

El modelo a probar es

$$\text{sales}_i = \beta_0 + \beta_1 \text{TV}_i + \beta_2 \text{radio}_i + \beta_3 \text{newspaper}_i + \varepsilon_i$$

Ajustando el modelo:

```
modelo_lm <- lm(sales ~ TV + radio + newspaper, data = Advertising)
summary(modelo_lm)

##
## Call:
## lm(formula = sales ~ TV + radio + newspaper, data = Advertising)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.8277 -0.8908  0.2418  1.1893  2.8292
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.938889   0.311908   9.422  <2e-16 ***
## TV           0.045765   0.001395  32.809  <2e-16 ***
## radio        0.188530   0.008611  21.893  <2e-16 ***
## newspaper   -0.001037   0.005871  -0.177    0.86
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.686 on 196 degrees of freedom
## Multiple R-squared:  0.8972, Adjusted R-squared:  0.8956
## F-statistic: 570.3 on 3 and 196 DF,  p-value: < 2.2e-16
```

Podemos considerar las funciones de pérdida MAE, MSE y RMSE, además, como desempeño el R^2

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}$$

```
y_hat_lm <- fitted(modelo_lm)
y_obs    <- Advertising$sales

cat("MSE:  ", round(Metrics::mse(y_obs, y_hat_lm), 4), "\n")
```

```
## MSE:    2.7841
```

```
cat("RMSE: ", round(Metrics::rmse(y_obs, y_hat_lm), 4), "\n")
```

```
## RMSE:   1.6686
```

```
cat("MAE:  ", round(Metrics::mae(y_obs, y_hat_lm), 4), "\n")
```

```
## MAE:    1.252
```

```
cat("R²:    ", round(summary(modelo_lm)$r.squared, 4), "\n")
```

```
## R²:     0.8972
```

2.3. Validación Cruzada

Aplicando los criterios vistos en el curso podemos estimar el error de predicción mediante validación cruzada con los distintos criterios:

2.3.1. Validación Cruzada Simple

Separamos la base aleatoriamente, nuevamente consideramos 70 % de los datos para entrenamiento:

```
set.seed(123)
n_ad <- nrow(Advertising)
idx_ad <- sample(1:n_ad, size = floor(0.7 * n_ad))
train_ad <- Advertising[idx_ad, ]
test_ad <- Advertising[-idx_ad, ]
```

Ajustamos con la data de entrenamiento:

```
mod_hold_lm <- lm(sales ~ TV + radio + newspaper, data = train_ad)
```

Estimamos el error de predicción y medimos el desempeño con la data de test:

```
y_pred_hold <- predict(mod_hold_lm, newdata = test_ad)

cat("--- Hold-Out 70/30 ---\n")
```

```
## --- Hold-Out 70/30 ---
```



```
cat("RMSE:", round(Metrics::rmse(test_ad$sales, y_pred_hold), 4), "\n")
```

```
## RMSE: 1.5236
```

```
cat("MAE: ", round(Metrics::mae(test_ad$sales, y_pred_hold), 4), "\n")
```

```
## MAE: 1.2886
```

```
cat("R²: ", round(cor(test_ad$sales, y_pred_hold)^2, 4), "\n")
```

```
## R²: 0.9223
```

2.3.2. Validación Cruzada K-Fold

Para esta estimación empleamos el paquete `caret`.

Seteamos los parámetros para el remuestreo: seleccionamos “Cross Validation” (`cv`), $K = 10$ pliegues:

```
set.seed(123)
ctrl_lm <- trainControl(method = "cv", number = 10,
                        savePredictions = "final")
```

Realizando el entrenamiento y estimando posteriormente el error:

```
mod_kfold_lm <- train(
  sales ~ TV + radio + newspaper,
  data      = Advertising,
  method    = "lm",
  trControl = ctrl_lm
)

kfold_lm_res <- mod_kfold_lm$resample
kable(kfold_lm_res, digits = 4,
      caption = "Métricas por pliegue - K-Fold (K=10, caret)",
      booktabs = TRUE)
```

Cuadro 3: Métricas por pliegue — K-Fold (K=10, caret)

RMSE	Rsquared	MAE	Resample
2.5769	0.8785	1.7967	Fold01
1.1578	0.9252	0.8608	Fold02
1.6926	0.9045	1.5048	Fold03
1.3763	0.9357	1.0692	Fold04
1.4665	0.9155	1.0548	Fold05
1.8338	0.8909	1.3097	Fold06
1.8324	0.8801	1.3617	Fold07
1.5303	0.9137	1.1338	Fold08
1.6195	0.9308	1.3008	Fold09
1.5619	0.9118	1.2950	Fold10

```
cat("\nPromedios K-Fold:\n")
```

```
##
## Promedios K-Fold:
```

```
colMeans(kfold_lm_res[, c("RMSE", "Rsquared", "MAE")]) %>%
  round(4) %>% print()
```

```
##      RMSE Rsquared      MAE
##    1.6648    0.9087    1.2687
```

2.3.3. Validación Cruzada Aleatoria (Bootstrap)

Seteamos semilla y cantidad de muestras bootstrap ($B = 200$):

```
set.seed(42)
B_lm <- 200
```

Calculamos los errores mediante validación cruzada bootstrap y promediamos:

```
err_boot_lm <- replicate(B_lm, {
  # Muestreando con reemplazo:
  idx_b <- sample(1:n_ad, size = n_ad, replace = TRUE)
  # Fase entrenamiento:
  train_b <- Advertising[idx_b, ]
  # Ajustamos modelo:
  mod_b <- lm(sales ~ TV + radio + newspaper, data = train_b)
  # Predecimos:
  y_pred_b <- predict(mod_b, newdata = Advertising)
  # Obtenemos el error:
  Metrics::mse(Advertising$sales, y_pred_b)
})

cat("Err_boot RMSE (estándar):", round(sqrt(mean(err_boot_lm)), 4), "\n")
```

```
## Err_boot RMSE (estándar): 1.6918
```

Podemos ver el error medio predicho es ligeramente mayor en este caso ($RMSE_{boot} = 1,69$) que en los anteriores ($RMSE_{simple} = 1,52$, $RMSE_{Kfold} = 1,66$), sin embargo sabemos que este valor presenta sesgo, podemos corregir esto empleando “Bootstrap 0,632”:

2.3.4. Estimador 0.632 $\widehat{Err}^{(0,632)}$

El estimador $\widehat{Err}^{(0,632)}$ corrige el sesgo del bootstrap:

$$\widehat{Err}^{(0,632)} = 0,368 \cdot \overline{err} + 0,632 \cdot \widehat{Err}^{(1)}$$

Empleamos directamente `caret` con `method = "boot632"`, que implementa este estimador de forma automática:

```

set.seed(42)
ctrl_boot632 <- trainControl(
  method      = "boot632",
  number      = 200,          # B = 200 muestras bootstrap
  savePredictions = "final"
)

mod_boot632_lm <- train(
  sales ~ TV + radio + newspaper,
  data      = Advertising,
  method    = "lm",
  trControl = ctrl_boot632
)

# Resultados
boot632_res <- mod_boot632_lm$results
cat("--- Bootstrap 0.632 ---\n")

```

```
## --- Bootstrap 0.632 ---
```

```
cat("RMSE:", round(boot632_res$RMSE, 4), "\n")
```

```
## RMSE: 1.7152
```

```
cat("MAE: ", round(boot632_res$MAE, 4), "\n")
```

```
## MAE: 1.2856
```

```
cat("R²: ", round(boot632_res$Rsquared, 4), "\n")
```

```
## R²: 0.8925
```

Vemos que realizando la corrección el error de predicción aumenta a $RMSE = 1,71$.

3. Ejercicios Propuestos

Ejercicio 1 Considere la base de datos `Weekly` [James:2021], la cual contiene 1089 observaciones semanales del mercado bursátil estadounidense entre 1990 y 2010. La variable de interés es `Direction`, que indica si el mercado subió (Up) o bajó (Down) en una semana determinada, en función de los retornos porcentuales rezagados `Lag1`, `Lag2`, `Lag3`, `Lag4`, `Lag5` y el volumen de transacciones `Volume`.

Proponga y ajuste un modelo de regresión logística, reporte las métricas de desempeño (Accuracy, Sensibilidad, Especificidad, F1 Score y AUC) y estime el error de predicción mediante validación cruzada simple, K-Fold con $K = 10$ y bootstrap $\widehat{\text{Err}}^{(0,632)}$.

Ejercicio 2 Considere la base de datos `Auto` [James:2021], la cual contiene información de 392 vehículos. La variable de interés es `mpg` (millas por galón), en función de `horsepower`, `weight`, `acceleration` y `year`. Proponga y ajuste un modelo de regresión múltiple, reporte las métricas de desempeño (RMSE, MAE y R^2) y estime el error de predicción mediante validación cruzada simple, K-Fold con $K = 10$ y bootstrap $\widehat{\text{Err}}^{(0,632)}$.

Referencias

- [James et al.(2021)] Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani (2021) *An Introduction to Statistical Learning with Applications in R*, 2nd edition. Springer Series in Statistics.
- [Hastie et al.(2008)] Trevor Hastie, Robert Tibshirani and Jerome Friedman (2008) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics.